

HEAVEN'S LIGHT IS OUR GUIDE



RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF MECHATRONICS ENGINEERING, 21Series

Course No: MTE 3104

Course Title: Microcontroller and Interfacing Sessional

Experiment No: 09

Experiment Name: 1 Hz signal generation using Atmega32 (Timer0) with Proteus Simulation.

Submitted To:

Md. Sakib Hassan

Lecturer,

Department of Mechatronics Engineering,
Rajshahi University of Engineering & Technology

Md Zunaid Hossen

Lecturer,

Department of Mechatronics Engineering,
Rajshahi University of Engineering & Technology

Submitted by:

Adiba Binta Newaz

2108043

Date of Submission:

21st June, 2025

Experiment No: 09

Experiment Name: 1 Hz signal generation using Atmega32 (Timer0) with Proteus Simulation.

Objectives:

1. To generate a signal of specified frequency using Timer0 of the ATmega32 microcontroller.
2. To understand the configuration and usage of Timer0 in the ATmega32 microcontroller for frequency generation.
3. To simulate the 1 Hz output in Proteus and validate the waveform using a virtual oscilloscope or by observing LED blinking.

Theory:

Timers of Atmega32 [1]: In the AVR ATmega16 / ATmega32 microcontrollers, there are three timers: Timer0, Timer1, and Timer2. These are 8bit, 16bit and 8bit respectively.

Key Registers of Timer0

1. **TCNT0: Timer / Counter Register 0** = It is an 8-bit register. It counts up with each pulse.
2. **TCCR0: Timer / Counter Control Register 0** = This is an 8-bit register used for the operation mode and the clock source selection

TCCR0 - Timer Control Resister

7	6	5	4	3	2	1	0
FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00

Table 1: Timer Control Register

TCCR0 Bits:

1. **Bit 7 - FOC0:** Force Compare Match: This write-only bit is used for forcing a compare match, useful in waveform generation.
2. **Bit 6, 3 - WGM00, WGM01:** Waveform Generation Mode

WGM00	WGM01	Timer0 mode selection bit
0	0	Normal
0	1	CTC
1	0	Phase correct PWM
1	1	Fast PWM

Table 2: Wave Generation Control Register

Timer Modes:

- **Normal Mode:** The timer counts from 0 to 255 and then overflows, resetting to 0. The overflow can trigger an interrupt.
- **CTC Mode (Clear timer on Compare Match):** The timer counts up to a value specified in the OCR0 (Output Compare Register 0) and then resets to 0. The compare match can trigger an interrupt.

- **PWM Mode:** Timer counts up and then down, providing a symmetric waveform. Useful for smoother motor control. The duty cycle is controlled by the value in OCR0.
- **Fast PWM Mode:** Timer counts up to a maximum value and then resets, providing a faster switching frequency.

3. Bit 5:4 - COM01:00: Compare Output Mode

These bits control the waveform generator. We will see this in the compare mode of the timer.

4. Bit 2:0 - CS02:CS00: Clock Source Select

These bits are used to select a clock source. When CS02: CS00 = 000, then timer is stopped. As it gets a value between 001 to 101, it gets a clock source and starts as the timer.

Table of Clock Source Select

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer / Counter stopped)
0	0	1	clk (no pre-scaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge.

Table 3: Clock Source Selection Configuration Table

Pre-scaler: The pre-scaler is a value that divides the clock frequency of the microcontroller to a lower frequency, allowing the timer to count at a slower rate. This is crucial for generating longer time periods and lower frequencies. Common pre-scaler values for Timer0 in ATmega32 are 1, 8, 64, 256, and 1024.

TIFR: Timer Interrupt Flag Register = TIFR is used to monitor the status of various timer interrupts.

7	6	5	4	3	2	1	0
OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0

Table 4: Timer Counter Interrupt Flag Register Table

TIFR Bits for Timer0:

- Bit 0 - TOV0: Timer0 Overflow Flag: Set when Timer0 overflows.
- Bit 1 - OCF0: Timer0 Output Compare Flag: Set when a compare match occurs

Frequency Calculation:

The frequency of the signal generated by the timer can be calculated using the following formula:

$$\text{Timer Overflow Frequency} = \frac{F_{CPU}}{\text{Prescaler} \times 256}$$

Here,

F_{CPU} = clock frequency of the microcontroller.

The timer counts from 0 to 255, thus 256 counts.

Required Software:

- Atmel Studio 7
- Proteus 8

Code:

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint8_t ovf_count = 0;

int main(void)
{
    DDRC = 0xFF;
    TIMSK |= 0x01;
    TCNT0 = 0;
    TCCR0 = 0x03;

    sei();

    while(1)
    {
    }

    ISR(TIMERO_OVF_vect){
        ovf_count = ovf_count + 1;

        if (ovf_count == 61)
        {
            PORTC ^= 0xFF;
            ovf_count = 0;
        }
    }
}
```

Code Explanation:

1. With Timer0 set to Normal (overflow) mode, this code creates a 1 Hz square wave signal on the ATmega32 microcontroller's PORTC. DDRC = 0xFF; initially sets all PORTC pins to outputs.
2. The timer is configured with TCCR0 = 0x03; choosing a prescaler of 64, and the timer count is started at zero with TCNT0 = 0;.
3. While sei(); initiates global interrupts.
4. TIMSK |= 0x01; enables the Timer0 overflow interrupt. After 256 counts, Timer0 overflows because it is an 8-bit timer. One overflow happens approximately every 16.384 milliseconds when using a 1 MHz clock and the prescaler, with each tick lasting 64 microseconds. To achieve a 1 Hz square wave, the code counts 61 overflows, which together take about 1 second ($61 \times 16.384 \text{ ms} \approx 1 \text{ second}$).
5. Inside the ISR(TIMERO_OVF_vect) interrupt service routine, a variable ovf_count is incremented every time the timer overflows. When this count reaches 61, the program toggles PORTC by executing PORTC ^= 0xFF; , flipping all the bits of PORTC to produce the square wave output. The overflow counter is then reset, and the process repeats to maintain the 1 Hz frequency.

Circuit Diagram:

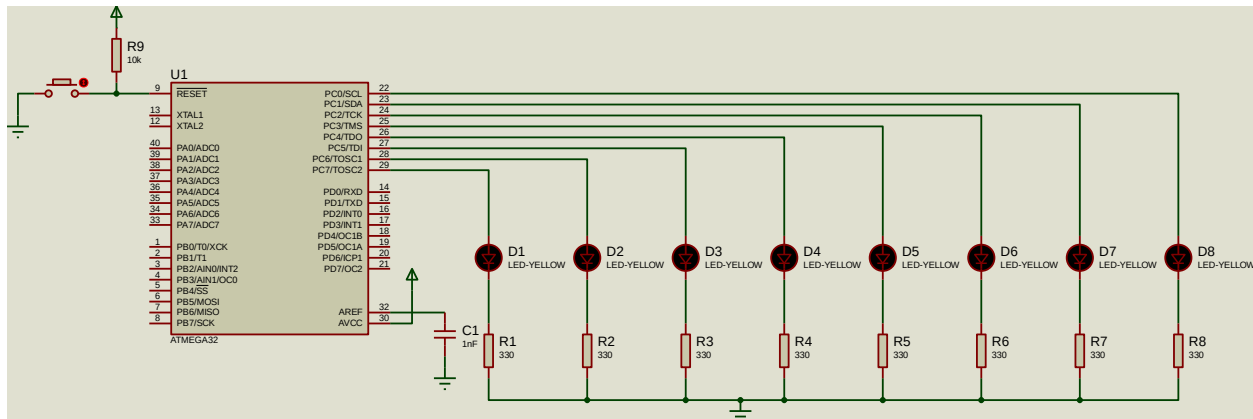


Figure 1: Circuit Diagram for showing the 1 Hz Signal in Proteus

Result:

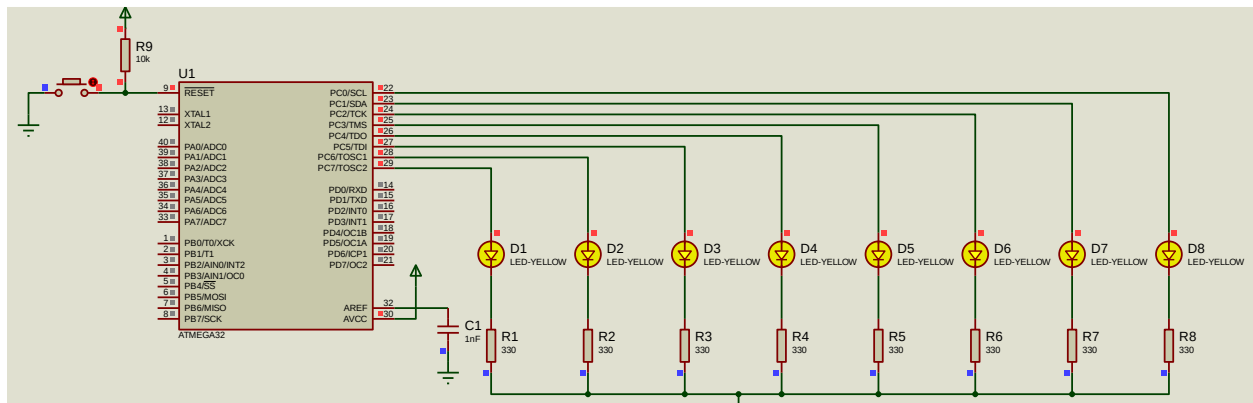


Figure 2: Simulation of the working 1 Hz signal

Discussion: The experiment explained how to use Timer0 in the ATmega32 microcontroller to generate a signal at a given frequency. Timer0 was set to Normal mode with the required pre-scaler settings, and overflow counts were calculated to generate correct frequencies. The selection of a pre-scaler and the calculation of overflow counts were regarded critical to reaching the desired output frequency.

Conclusion: This experiment successfully achieves its aims. This program explains how to use the AVR Timer0 and interrupts to accomplish a periodic operation, specifically, switching PORTC every second. Such approaches are critical in embedded systems where accurate timing operations are required. The usage of interrupts guarantees that the microcontroller can execute other activities concurrently without being hampered by timing checks, hence improving overall performance.

References:

[1] "Timer in AVR ATmega16/ATmega32," Electronicwings.com. [Online]. Available: <https://www.electronicwings.com/avr-atmega/atmega1632-timer>. [Accessed: 20-Jun-2025]